PocketMind Prerequisites — AI & LLM Terms and Techniques

🧭 **Purpose:** A beginner-friendly prerequisite guide for building PocketMind. Learn the sections in order; do not wait to master every mathematical proof before coding. The target is enough understanding to explain each component, implement a small version, test it, and debug it.

Recommended approach: 40% concepts · 50% coding · 10% notes and review

Completion standard: You can define the term in your own words, identify its tensor shapes, and implement or test a minimal example.

---

1. The mental model

A language model repeatedly performs one task:

```
Previous tokens → probability distribution for the next token
```

A complete LLM system adds several layers around that task:

```
Raw text
→ cleaning
→ tokenization
→ training examples
→ neural network
→ loss
→ backpropagation
→ optimization
→ checkpoint
→ decoding
→ retrieval/tools
→ evaluation and safety
```

Core distinctions

- **Artificial intelligence (AI):** Broad field of building systems that perform tasks associated with intelligence.
- **Machine learning (ML):** AI systems that learn patterns from data rather than relying only on manually written rules.
- **Deep learning:** ML based on neural networks with many representation-learning layers.
- **Natural language processing (NLP):** Computing techniques for understanding and generating human language.
- **Language model (LM):** A model that assigns probabilities to token sequences or predicts tokens.
- **Large language model (LLM):** A language model with sufficiently large data, parameter count, and capabilities. “Large” has no fixed threshold.
- **Generative AI:** Models that produce new text, images, audio, code, or other content.
- **Foundation model:** A broadly trained model intended to support multiple downstream tasks.
- **Model:** The mathematical function and learned parameters.
- **AI system:** The model plus tokenizer, data pipeline, retrieval, tools, policies, interface, and monitoring.
- **Training:** Adjusting model parameters using examples and an objective.
- **Inference:** Using a trained model to produce outputs without updating its parameters.

💡 PocketMind is an **AI system** built around a small language model. Its practical ability comes from the combination of model, retrieval, constrained actions, and deterministic software—not model size alone.

---

2. Prerequisite programming knowledge

2.1 Python fundamentals

Know these before implementing the model:

- **Variable:** A name bound to a value.
- **Type:** The kind of value, such as integer, float, string, list, or tensor.
- **Function:** Reusable computation with inputs and outputs.
- **Class:** A structure combining data and behavior.
- **Dataclass:** A concise Python class primarily used to store typed data.
- **Module/package:** A file or collection of files that organizes reusable code.
- **Iterator/generator:** Produces values lazily rather than storing all values at once.
- **Context manager:** Controls setup and cleanup, used by `with` blocks.
- **Exception:** A structured runtime error that can be raised and handled.
- **Type hint:** Annotation describing expected types.
- **Virtual environment:** Isolated Python dependency environment.
- **Dependency lockfile:** Records exact dependency versions for reproducibility.
- **CLI:** Command-line interface.
- **Serialization:** Converting an object into a storable or transferable representation.

2.2 Python techniques to practice

- Reading text and binary files.
- Writing classes with typed methods.

- Using `pathlib` safely.
- Writing command-line programs.
- Loading YAML and JSON.
- Logging structured metrics.
- Writing unit tests with `pytest`.
- Profiling execution time and memory.
- Using iterators to avoid loading entire datasets into RAM.

2.3 NumPy fundamentals

- **Array:** A multidimensional block of values of one data type.
- **Shape:** Size of each array dimension.
- **Rank/ndim:** Number of dimensions.
- **Axis:** Dimension along which an operation is performed.
- **Dtype:** Numeric representation such as `float32`, `float16`, or `uint16`.
- **Indexing/slicing:** Selecting array elements or ranges.
- **Broadcasting:** Automatically expanding compatible dimensions during operations.
- **Vectorization:** Replacing Python loops with efficient array operations.
- **Memory mapping:** Accessing a disk-backed array as if it were in memory.
- **Contiguous memory:** Values stored in an order suitable for efficient operations.

2.4 Git and engineering basics

- **Repository:** Version-controlled project directory.
- **Commit:** Saved snapshot of changes.
- **Branch:** Independent line of development.
- **Diff:** Representation of what changed.
- **Tag:** Named reference, commonly used for releases.
- **Semantic versioning:** Version format such as `1.2.0` representing major, minor, and patch changes.

- **Continuous integration:** Automated tests and checks run after changes.
- **Regression:** Previously working behavior that breaks after a change.

Readiness exercise

- [ ] Build a Python CLI that reads a large text file lazily.
- [ ] Count lines and UTF-8 bytes without loading the complete file.
- [ ] Save the results as JSON.
- [ ] Write tests for empty, Unicode, and missing files.

---

3. Mathematics you actually need

3.1 Scalars, vectors, matrices, and tensors

- **Scalar:** One number, such as a loss value.
- **Vector:** One-dimensional sequence of numbers.
- **Matrix:** Two-dimensional grid of numbers.
- **Tensor:** General multidimensional array.
- **Element:** A single tensor value.
- **Dimension:** One direction of a tensor.
- **Shape:** Length of every dimension.

Common language-model shapes:

```
B = batch size
T = sequence length
D = model dimension
V = vocabulary size
H = number of attention heads
Dh = head dimension

input_ids       [B, T]
embeddings      [B, T, D]
queries         [B, H, T, Dh]
attention       [B, H, T, T]
logits          [B, T, V]
```

3.2 Linear algebra

- **Dot product:** Multiplies corresponding vector values and sums them; measures aligned magnitude.
- **Matrix multiplication:** Combines rows and columns to perform learned linear transformations.
- **Transpose:** Reorders matrix axes; attention uses K^T.
- **Identity matrix:** Matrix that leaves vectors unchanged when multiplied.
- **Norm:** Measurement of vector magnitude.
- **L1 norm:** Sum of absolute values.
- **L2 norm:** Square root of the sum of squared values.
- **Cosine similarity:** Angle-based similarity between vectors.
- **Linear transformation:** $y = xW$.
- **Affine transformation:** $y = xW + b$.
- **Projection:** Learned mapping into another vector space.
- **Rank:** Number of independent directions represented by a matrix; different from tensor `ndim`.

3.3 Probability and statistics

- **Probability distribution:** Possible outcomes and their probabilities.
- **Categorical distribution:** Distribution over discrete choices, such as vocabulary tokens.
- **Logit:** Unnormalized score before softmax.
- **Softmax:** Converts logits into probabilities that sum to one.
- **Log probability:** Natural logarithm of probability; numerically convenient.
- **Expectation/mean:** Probability-weighted average.
- **Variance:** Average squared distance from the mean.
- **Standard deviation:** Square root of variance.
- **Sampling:** Selecting an outcome according to probabilities.
- **Entropy:** Uncertainty of a distribution.
- **Cross-entropy:** Measures how poorly predicted probabilities match target outcomes.

- **Likelihood:** Probability assigned to observed data by a model.
- **Maximum likelihood estimation:** Choosing parameters that maximize training-data likelihood.
- **Independent and identically distributed (IID):** Simplifying assumption that examples are independent and share a distribution; language data often violates it.
- **Distribution shift:** Deployment data differs from training data.

Softmax:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Cross-entropy for the correct token `y`:

$$L = -\log p_y$$

3.4 Calculus and optimization

- **Function:** Maps inputs to outputs.
- **Derivative:** Rate of change of a function.
- **Partial derivative:** Derivative with respect to one input while others are held fixed.
- **Gradient:** Vector of partial derivatives with respect to parameters.
- **Chain rule:** Combines derivatives through a sequence of operations.
- **Computational graph:** Graph of tensor operations used to compute outputs and gradients.
- **Loss surface:** Loss value as a function of all parameters.
- **Gradient descent:** Updates parameters opposite the gradient.
- **Learning rate:** Size of an optimization update.
- **Local minimum:** Region where nearby parameter choices have higher loss.
- **Saddle point:** Flat or mixed-curvature region that is not a true minimum.

Basic update:

$$\theta_{new} = \theta_{old} - \eta \nabla_\theta L$$

3.5 Numerical stability

- **Floating-point number:** Approximate computer representation of a real number.
- **Precision:** Number of reliably represented bits/digits.
- **Overflow:** Value becomes too large for the dtype.
- **Underflow:** Small value becomes zero or loses precision.
- **NaN:** Invalid numeric result.
- **Infinity:** Value outside finite range.
- **Epsilon:** Small constant used to avoid division by zero.
- **Stable softmax:** Subtracts the maximum logit before exponentiation.
- **Log-sum-exp trick:** Stable computation for logarithms of exponent sums.

Math readiness exercises

- [ ] Implement vector dot product using loops, then NumPy.
- [ ] Implement stable softmax.
- [ ] Compute cross-entropy for one target token.
- [ ] Track the shapes in QK^T attention by hand.
- [ ] Derive the gradient of $y = wx + b$ for squared error.

---

4. PyTorch vocabulary and techniques

4.1 Core terms

- **Tensor:** PyTorch multidimensional numeric array.
- **Device:** Location of a tensor, usually CPU or CUDA device.
- **CUDA:** NVIDIA platform used for GPU computation.
- **Kernel:** Low-level function executed on a device.
- **Autograd:** PyTorch automatic differentiation system.
- **Parameter:** Tensor registered for gradient-based optimization.
- **Module:** Reusable neural-network component derived from `nn.Module`.

- **Forward pass:** Computation from input to prediction and loss.
- **Backward pass:** Computation of gradients from loss back through the graph.
- **Gradient:** Parameter direction and magnitude that increases loss.
- **Optimizer:** Algorithm that applies gradients to parameters.
- **State dict:** Mapping containing model or optimizer state.
- **Training mode:** Enables training-specific behavior such as dropout.
- **Evaluation mode:** Disables training-only behavior.
- **No-grad/inference mode:** Prevents gradient graph construction during inference.
- **DataLoader:** Batches and supplies dataset examples.

4.2 Essential operations

- `view`, `reshape`, `transpose`, `permute`, `unsqueeze`, `squeeze`.
- `matmul`, `einsum`, `softmax`, `argmax`, `topk`.
- Boolean masks and `masked_fill`.
- Concatenation and stacking.
- Gather and scatter.
- Detaching a tensor from autograd.
- Moving tensors between devices.
- Saving and loading state safely.

4.3 Training-specific techniques

- **Mixed precision:** Uses lower-precision arithmetic where safe to reduce memory and increase speed.
- **FP32:** 32-bit floating point; reliable baseline.
- **FP16:** 16-bit floating point with limited exponent range.
- **BF16:** 16-bit format with FP32-like exponent range.
- **Autocast:** Automatically selects operation precision.
- **Gradient scaler:** Prevents FP16 gradient underflow by scaling loss.

- **Gradient accumulation:** Combines gradients from multiple micro-batches before an optimizer step.
- **Activation checkpointing:** Recomputes forward activations during backward to save memory.
- **Gradient clipping:** Limits gradient norm to reduce unstable updates.
- **Fused operation:** Multiple operations combined into a faster kernel.
- **Compilation:** Graph optimization through tools such as `torch.compile`.

PyTorch readiness exercise

- [ ] Build `Linear → ReLU → Linear` without `nn.Sequential`.
- [ ] Train it on a small classification dataset.
- [ ] Save and reload it.
- [ ] Confirm reloaded logits match.
- [ ] Run the same model in mixed precision and compare outputs.

---

5. Machine-learning foundations

5.1 Data terminology

- **Example/sample:** One training item.
- **Feature:** Input information used by a model.
- **Label/target:** Desired output.
- **Dataset:** Collection of examples.
- **Training set:** Used to update parameters.
- **Validation set:** Used for model selection and tuning.
- **Test set:** Used once for final unbiased evaluation.
- **Batch:** Multiple examples processed together.
- **Micro-batch:** Examples processed in one forward/backward pass.
- **Effective batch:** Total examples contributing to one optimizer update.
- **Epoch:** One pass through the training set.
- **Step/iteration:** Usually one optimizer update.

- **Token budget:** Total number of tokens processed during training.
- **Data leakage:** Validation/test information unintentionally reaches training.
- **Contamination:** Evaluation examples appear in training data.

5.2 Learning behavior

- **Generalization:** Performance on unseen examples.
- **Memorization:** Reproduction of specific training patterns.
- **Underfitting:** Model is too limited or insufficiently trained.
- **Overfitting:** Training performance improves while unseen-data performance degrades.
- **Bias:** Systematic error from model assumptions or data.
- **Variance:** Sensitivity to the specific training sample.
- **Regularization:** Technique that discourages brittle overfitting.
- **Dropout:** Randomly zeros activations during training.
- **Weight decay:** Penalizes or shrinks parameter magnitudes.
- **Early stopping:** Stops training when validation performance stops improving.
- **Hyperparameter:** User-chosen setting not learned directly, such as learning rate.
- **Ablation:** Controlled removal/change of one component to measure its contribution.
- **Baseline:** Simpler reference system used for comparison.

5.3 Objectives and optimization

- **Objective/loss function:** Scalar measurement the optimizer minimizes.
- **Mini-batch gradient descent:** Estimates gradients using a batch.
- **SGD:** Basic stochastic gradient-descent optimizer.
- **Momentum:** Moving average of past gradients.
- **Adam:** Adaptive optimizer using first and second gradient moments.
- **AdamW:** Adam with decoupled weight decay.

- **Learning-rate schedule:** Changes learning rate during training.
- **Warmup:** Gradually increases learning rate at the start.
- **Cosine decay:** Smoothly lowers learning rate following a cosine curve.
- **Optimizer state:** Additional values such as momentum estimates.
- **Gradient norm:** Overall gradient magnitude.

5.4 Reproducibility

- **Random seed:** Initial value controlling pseudorandom operations.
- **Determinism:** Same inputs and setup produce the same result.
- **Checkpoint:** Saved model and training state.
- **Resume:** Continue from a checkpoint.
- **Experiment tracking:** Recording code, configuration, data, metrics, and artifacts.
- **Artifact:** Produced file such as tokenizer, checkpoint, or evaluation report.

---

6. Language-model fundamentals

6.1 Sequence terminology

- **Corpus:** Collection of text used for training or evaluation.
- **Sequence:** Ordered tokens.
- **Vocabulary:** Set of tokens the model can represent.
- **Token ID:** Integer assigned to a token.
- **Context:** Previous tokens available to the model.
- **Context length/window:** Maximum number of tokens processed together.
- **Next-token prediction:** Predicting token `t+1` from tokens up to `t`.
- **Autoregressive model:** Generates one token at a time conditioned on previous tokens.
- **Causal model:** Prevents information from future tokens reaching past positions.

- **Teacher forcing:** During training, supplies the true previous tokens rather than model-generated tokens.
- **Sequence packing:** Combines documents efficiently into fixed-length training sequences.
- **Document boundary:** Marker preventing unrelated documents from being treated as continuous text.

6.2 Model outputs

- **Hidden state:** Internal vector representation at a sequence position.
- **Logits:** Score for every possible next token.
- **Probability:** Softmax-normalized token likelihood.
- **Language-model head:** Final projection from hidden dimension to vocabulary dimension.
- **Weight tying:** Shares token-embedding and output-projection weights.
- **Negative log-likelihood:** Loss obtained by penalizing low probability on true tokens.
- **Perplexity:** Exponentiated average cross-entropy; lower usually means better next-token prediction.

$$Perplexity = e^{average\ cross\ entropy}$$

6.3 Pretraining and adaptation

- **Pretraining:** Broad next-token training on a large corpus.
- **Fine-tuning:** Additional training for a domain or task.
- **Supervised fine-tuning (SFT):** Training on input–desired-output examples.
- **Instruction tuning:** SFT designed to improve instruction following.
- **Preference optimization:** Trains from comparisons between preferred and rejected answers.
- **RLHF:** Reinforcement learning from human feedback.
- **DPO:** Direct Preference Optimization; preference learning without a conventional RL loop.
- **LoRA:** Low-rank trainable adapters applied to frozen weights.

- **PEFT:** Parameter-efficient fine-tuning family that updates a small parameter subset.

PocketMind v1 primarily uses pretraining, synthetic supervised examples, retrieval training, and correction replay. RLHF is not required.

---

7. Tokenization terms and techniques

- **Character tokenization:** One token per character.
- **Byte tokenization:** One token per byte; supports any input without unknown characters.
- **Word tokenization:** Splits text into words; poor handling of rare words and code.
- **Subword tokenization:** Represents common text fragments as tokens.
- **BPE:** Byte Pair Encoding; repeatedly merges frequent adjacent symbols.
- **Byte-backed BPE:** Starts from all bytes, ensuring every input is representable.
- **Unigram tokenizer:** Chooses subword pieces using a probabilistic vocabulary model.
- **Special token:** Reserved control marker such as `<BOS>`.
- **BOS/EOS:** Beginning/end-of-sequence markers.
- **PAD:** Padding marker for equal-length batches.
- **UNK:** Unknown token.
- **Encode:** Convert text into token IDs.
- **Decode:** Convert token IDs back into text.
- **Merge rule:** BPE rule combining two symbols.
- **Token fertility:** Number of tokens needed to represent a word or text span.
- **Vocabulary size:** Number of token types.
- **Tokenizer compression ratio:** Bytes or characters represented per token.
- **Protected token:** Special token excluded from ordinary merge behavior.
- **Round trip:** Decode of encoded text reproduces the original text.

Important trade-off

A larger vocabulary shortens sequences but increases embedding/output parameters. A smaller vocabulary uses fewer embedding parameters but produces longer sequences, increasing sequence-computation cost.

Required practical techniques

- Train tokenizer only on training data.
- Freeze and version tokenizer before model training.
- Preserve special tokens as indivisible units.
- Test Unicode, code, JSON, whitespace, and file paths.
- Record tokenizer hash in every checkpoint.

---

8. Neural-network building blocks

8.1 Parameters and representations

- **Neuron/unit:** One computed activation value.
- **Layer:** Transformation from one representation to another.
- **Parameter/weight:** Learned numeric value.
- **Bias:** Learned additive value.
- **Embedding:** Learned vector associated with a discrete ID.
- **Token embedding:** Vector representing a token.
- **Position representation:** Information indicating token order.
- **Model dimension (`d_model`):** Width of the main hidden vectors.
- **Parameter count:** Number of learned scalar values.
- **Initialization:** Starting parameter values before training.
- **Xavier/Glorot initialization:** Variance-aware initialization for linear layers.
- **Kaiming/He initialization:** Initialization designed for rectified activations.

8.2 Activations

- **Activation function:** Nonlinear transformation allowing complex functions.

- **ReLU:** `max(0, x)`.
- **GELU:** Smooth activation common in Transformers.
- **SiLU/Swish:** `x × sigmoid(x)`.
- **Sigmoid:** Maps values into `(0, 1)`; useful for gates.
- **Tanh:** Maps values into `(-1, 1)`.
- **Saturation:** Region where gradients become very small.

8.3 Normalization and residuals

- **Normalization:** Stabilizes activation scale.
- **LayerNorm:** Normalizes hidden features using mean and variance.
- **RMSNorm:** Normalizes using root-mean-square without subtracting the mean.
- **Pre-norm:** Normalization before attention or feed-forward sublayer.
- **Post-norm:** Normalization after the residual addition.
- **Residual/skip connection:** Adds a block's input to its output.
- **Residual stream:** Main hidden representation flowing through the model.
- **Epsilon:** Small normalization constant.

8.4 Feed-forward networks

- **MLP/FFN:** Per-token neural network, usually expansion → activation → contraction.
- **Expansion ratio:** `ffn_dim / d_model`.
- **Gated linear unit:** Multiplies a value projection by a learned gate.
- **SwiGLU:** Gated FFN using SiLU activation.

---

9. Attention and Transformer terminology

9.1 Attention intuition

Attention allows each token to gather information from other allowed token positions.

- **Query (Q):** What the current position is looking for.

- **Key (K):** What each available position offers for matching.
- **Value (V):** Information copied when a key receives attention.
- **Attention score:** Similarity between query and key.
- **Attention weight:** Softmax-normalized score.
- **Scaled dot-product attention:** Divides `QKᵀ` by square root of head dimension.
- **Self-attention:** Q, K, and V come from the same sequence.
- **Cross-attention:** Query comes from one sequence and K/V from another.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}} + mask\right)V$$

9.2 Masks

- **Attention mask:** Controls which positions can be attended to.
- **Causal mask:** Blocks future positions.
- **Padding mask:** Blocks padding tokens.
- **Local/sliding-window attention:** Allows attention only within a nearby window.
- **Full attention:** Every position can attend to every allowed position.
- **Future-token leakage:** Bug where training output accesses future targets.

9.3 Multi-head attention

- **Attention head:** Independent Q/K/V subspace.
- **Number of heads:** Count of parallel attention patterns.
- **Head dimension:** Hidden width assigned to each head.
- **Concatenation:** Combines head outputs.
- **Output projection:** Mixes concatenated head information.
- **Grouped-query attention:** Shares K/V heads across query heads to reduce inference memory.
- **Multi-query attention:** Uses one shared K/V head for all query heads.

9.4 Positions and caching

- **Absolute positional embedding:** Learned or fixed vector for each position.
- **Sinusoidal position encoding:** Fixed sine/cosine position features.
- **RoPE:** Rotary Position Embedding; rotates Q/K components according to position.
- **Relative position:** Represents distances between tokens.
- **KV cache:** Stores previous keys and values during autoregressive generation.
- **Prefill:** Initial processing of the complete prompt.
- **Decode step:** Generation of one subsequent token using cached state.

9.5 Transformer block

```
x
→ norm
→ attention
→ residual addition
→ norm
→ feed-forward network
→ residual addition
```

- **Encoder:** Processes input bidirectionally; common in classification and embedding models.
- **Decoder:** Uses causal attention for generation.
- **Encoder–decoder:** Encoder processes source; decoder generates output with cross-attention.
- **Decoder-only Transformer:** Architecture used by GPT-like autoregressive models.

Attention readiness exercises

- [ ] Implement one attention head with explicit tensor operations.
- [ ] Print every intermediate shape.
- [ ] Visualize the causal mask.
- [ ] Change a future token and verify earlier outputs remain identical.

- [ ] Compare reference attention with optimized attention.

10. Recurrent and memory techniques

- **Recurrent neural network (RNN):** Processes a sequence while carrying hidden state.
- **Hidden state:** Compressed memory passed between sequence steps.
- **Recurrence:** Reuse of the same transformation across positions.
- **Vanishing gradient:** Gradient becomes too small across many recurrent steps.
- **Exploding gradient:** Gradient becomes excessively large.
- **Gate:** Learned value controlling information flow.
- **LSTM:** Gated recurrent architecture with cell and hidden states.
- **GRU:** Simpler gated recurrent architecture.
- **State-space model (SSM):** Sequence model based on evolving a state with structured dynamics.
- **Selective state-space model:** Allows input-dependent state updates.
- **Truncated backpropagation through time:** Limits recurrent gradient history to bounded chunks.
- **State leakage:** Hidden state from one document or user incorrectly influences another.
- **Chunked scan:** Parallel or blockwise computation of recurrence.

PocketMind's recurrent-memory layer is an educational hybrid component. It must be compared with an equal-parameter attention baseline before claiming improvement.

11. Training mechanics in detail

11.1 One training iteration

1. Load token sequence.
2. Create input and one-token-shifted target.

3. Run forward pass.
4. Compute cross-entropy.
5. Run backward pass.
6. Clip gradients if required.
7. Apply optimizer update.
8. Update learning-rate scheduler.
9. Record metrics.
10. Periodically validate and checkpoint.

11.2 Batch terminology

- **Batch size:** Examples contributing to a computation.
- **Micro-batch size:** Examples that fit in one device pass.
- **Gradient accumulation steps:** Number of micro-batches before an update.
- **Effective batch size:** `micro_batch × accumulation × device count`.
- **Sequence length:** Tokens in each example.
- **Tokens per step:** Effective batch multiplied by sequence length.

11.3 Stability techniques

- Validate data before device transfer.
- Start with float32 for debugging.
- Monitor loss, gradients, and non-finite values.
- Use warmup and clipping.
- Overfit one tiny batch before scaling.
- Evaluate with dropout disabled and gradients off.
- Save atomic checkpoints.
- Resume with random states restored.

11.4 Common failure terms

- **NaN loss:** Invalid numerical result.
- **Divergence:** Loss becomes unstable or continually increases.

- **Loss plateau:** Loss stops improving.
- **Gradient explosion:** Gradient norm becomes extremely large.
- **Dead activation:** Unit produces uninformative output.
- **Memory leak:** Allocated memory grows unexpectedly.
- **OOM:** Out-of-memory error.
- **Bottleneck:** Component limiting speed.
- **Throughput:** Tokens processed per second.
- **Latency:** Time required for one operation or response.

---

12. Data engineering terms and techniques

- **Data provenance:** Record of where data came from.
- **License:** Terms governing permitted data/code use.
- **Manifest:** Machine-readable record of sources and transformations.
- **Hash/checksum:** Content fingerprint used for integrity and deduplication.
- **Normalization:** Converting equivalent representations into a consistent form.
- **Deduplication:** Removing duplicate content.
- **Near-duplicate:** Content that differs slightly but is substantially the same.
- **MinHash:** Approximate method for comparing document-set similarity.
- **Shingle:** Consecutive token or character group used for similarity.
- **Filtering:** Excluding low-quality, unsafe, or unsuitable examples.
- **Secret scanning:** Detecting credentials and private keys.
- **Synthetic data:** Programmatically generated examples.
- **Hard negative:** Incorrect example that appears highly relevant.
- **Curriculum learning:** Training from easier examples toward harder ones.
- **Data mixture:** Proportion assigned to each data category.
- **Sampling weight:** Probability of selecting a data category or example.
- **Data loader worker:** Process/thread preparing batches.

- **Memory-mapped dataset:** Disk-backed tokens accessed without full RAM loading.

Essential techniques

- Split by complete document or repository.
- Deduplicate before splitting when possible.
- Keep a held-out test set untouched.
- Version transformations and manifests.
- Inspect random samples after every pipeline stage.
- Reject generated synthetic targets that fail schema validation.

---

13. Generation and decoding

- **Decoding:** Selecting output tokens from logits.
- **Greedy decoding:** Always chooses the highest-probability token.
- **Temperature:** Divides logits before softmax; lower is more deterministic.
- **Top-k sampling:** Samples only from the `k` highest-logit tokens.
- **Top-p/nucleus sampling:** Samples from the smallest set whose cumulative probability reaches `p`.
- **Beam search:** Maintains several high-scoring output sequences.
- **Stop token:** Token that ends generation.
- **Maximum new tokens:** Hard generation-length limit.
- **Repetition penalty:** Adjusts logits to discourage repeated tokens.
- **Degeneration:** Repetitive, incoherent, or collapsed generated text.
- **Deterministic decoding:** Same prompt and settings produce the same output.
- **Constrained decoding:** Masks tokens that violate a grammar or schema.
- **Grammar:** Rules defining valid output sequences.
- **Parser state:** Current structural position while reading generated output.
- **Token mask:** Set of token IDs permitted at a generation step.

For PocketMind:

- Use sampling for conversational text.
- Use greedy constrained decoding for actions.
- Validate final output again even after constrained generation.
- Never treat a model-generated action as authorization.

---

14. Retrieval and memory systems

14.1 Retrieval concepts

- **Information retrieval (IR):** Finding relevant documents for a query.
- **Index:** Data structure supporting fast search.
- **Chunk:** Searchable portion of a document.
- **Chunk overlap:** Repeated boundary text between adjacent chunks.
- **Lexical retrieval:** Matches exact or related written terms.
- **Inverted index:** Maps terms to documents containing them.
- **TF:** Term frequency within a document.
- **IDF:** Gives more weight to terms rare across documents.
- **TF-IDF:** Weighted vector representation based on term importance.
- **BM25:** Strong lexical ranking algorithm.
- **FTS:** Full-text search.
- **Embedding model:** Maps text into vectors.
- **Dense retrieval:** Retrieves by embedding similarity.
- **Sparse retrieval:** Retrieves using mostly-zero lexical vectors.
- **Vector database:** Stores and searches vectors.
- **Nearest-neighbor search:** Finds vectors most similar to a query.
- **ANN:** Approximate nearest-neighbor search for speed at scale.
- **Hybrid retrieval:** Combines lexical and dense scores.
- **Reranker:** Re-scores a small candidate set more accurately.

- **Top-k:** Number of highest-ranked results returned.

14.2 Retrieval-augmented generation

- **RAG:** Retrieves external evidence and places it in model context before generation.
- **Grounding:** Basing an answer on supplied evidence.
- **Citation/source attribution:** Identifying the evidence location.
- **Abstention:** Saying evidence is insufficient rather than guessing.
- **Retrieval recall:** Whether relevant evidence appears among retrieved results.
- **Context poisoning:** Malicious or misleading instructions inside retrieved content.
- **Prompt injection:** Untrusted text attempts to override trusted instructions.

14.3 Contrastive learning

- **Positive pair:** Query and relevant document.
- **Negative pair:** Query and irrelevant document.
- **Hard negative:** Irrelevant document that resembles a relevant one.
- **Similarity score:** Numeric relevance estimate.
- **Temperature (τ):** Controls contrastive-softmax sharpness.
- **In-batch negatives:** Other documents in the batch serve as negatives.
- **Embedding normalization:** Scales vectors to unit length before cosine similarity.

Retrieval readiness exercise

- [ ] Index a small documentation folder.
- [ ] Implement TF-IDF or BM25 retrieval.
- [ ] Create 20 questions with known source passages.
- [ ] Calculate Recall@1 and Recall@5.
- [ ] Inspect every failed retrieval manually.

15. Structured actions and tool use

- **Tool/function calling:** Model emits a structured request for deterministic software.
- **Schema:** Formal definition of valid fields and values.
- **JSON Schema:** Standard vocabulary for describing JSON structure.
- **Validation:** Checking data against rules.
- **Pydantic model:** Typed Python validation model.
- **Discriminated union:** Selects a schema variant using a field such as `action`.
- **Strict parsing:** Rejects unknown or incorrectly typed fields.
- **Orchestrator:** Coordinates model, retrieval, tools, and responses.
- **Executor:** Deterministic component that performs an approved operation.
- **Policy:** Rules defining permitted actions.
- **Allowlist:** Explicitly permitted set.
- **Denylist:** Explicitly prohibited set; generally weaker as the only defense.
- **Sandbox:** Restricted execution environment.
- **Least privilege:** Give each component only the access it requires.
- **Path traversal:** Attempt to escape a permitted directory using paths such as `../`.
- **Symlink:** Filesystem pointer that can bypass naive path checks.
- **Timeout:** Maximum operation duration.
- **Output bound:** Maximum returned data size.
- **Human confirmation:** Explicit approval before a consequential action.

Core rule:

```
Model proposes → validator checks → policy authorizes → executor performs
```

---

16. Evaluation terminology

16.1 General metrics

- **Metric:** Numeric measurement of performance.
- **Benchmark:** Fixed set of cases and evaluation rules.
- **Accuracy:** Fraction of correct predictions.
- **Precision:** Correct positive predictions divided by all positive predictions.
- **Recall:** Correct positive predictions divided by all actual positives.
- **F1 score:** Harmonic mean of precision and recall.
- **Exact match:** Prediction equals the target exactly.
- **Calibration:** Predicted confidence corresponds to observed correctness.
- **Confidence interval:** Range expressing uncertainty in a metric estimate.
- **Statistical significance:** Evidence that a difference is unlikely due to random variation.
- **Qualitative evaluation:** Structured human inspection rather than only numeric scores.

16.2 Retrieval metrics

- **Recall@k:** Fraction of queries whose relevant result appears in top `k`.
- **Precision@k:** Relevant fraction among top `k` results.
- **MRR:** Mean reciprocal rank of the first relevant result.
- **NDCG:** Ranking metric that rewards relevant items near the top.

16.3 Generation metrics

- **Perplexity:** Next-token prediction quality.
- **Repetition rate:** Frequency of repeated text patterns.
- **Unsupported-claim rate:** Fraction of claims not backed by supplied evidence.
- **Abstention accuracy:** Whether the model declines when evidence is insufficient.
- **Schema-valid rate:** Fraction of outputs accepted by the schema.
- **Action-type accuracy:** Fraction selecting the correct action.
- **Field-level F1:** Correctness of individual structured fields.

16.4 Evaluation discipline

- Define metrics before major experiments.
- Keep test data untouched until release evaluation.
- Report speed and memory with quality.
- Compare against simple baselines.
- Preserve failed examples, not only averages.
- Repeat important experiments across multiple seeds where practical.

---

17. Efficiency and systems terminology

- **RAM:** General system memory.
- **VRAM:** Memory located on the GPU.
- **Bandwidth:** Rate at which data moves between memory and compute.
- **Compute-bound:** Limited mainly by arithmetic capacity.
- **Memory-bound:** Limited mainly by memory transfer.
- **Parameter memory:** Storage for model weights.
- **Activation memory:** Intermediate tensors saved for backward.
- **Gradient memory:** Storage for parameter gradients.
- **Optimizer memory:** Additional optimizer states.
- **KV-cache memory:** Stored keys/values during generation.
- **Quantization:** Represents values with fewer bits.
- **INT8/INT4:** 8-bit or 4-bit integer representations.
- **Post-training quantization:** Quantizes an already trained model.
- **Quantization-aware training:** Simulates quantization during training.
- **Pruning:** Removes weights or structures judged unnecessary.
- **Knowledge distillation:** Trains a smaller student from a larger teacher.
- **FlashAttention:** Memory-efficient exact attention algorithm.
- **Profiling:** Measuring where time and memory are spent.
- **Kernel launch overhead:** Cost of starting device operations.

- **Data-transfer overhead:** Cost of moving values between CPU and GPU.

> ⚙️ Optimization order: make it correct, test it, profile it, then optimize the measured bottleneck. Do not begin with quantization, compilation, or custom kernels.

---

18. Security and responsible-AI terms

- **Threat model:** Description of assets, attackers, risks, and defenses.
- **Attack surface:** All places where untrusted input can affect the system.
- **Prompt injection:** Content attempts to control model behavior against trusted policy.
- **Indirect prompt injection:** Malicious instruction arrives through a retrieved file or webpage.
- **Data poisoning:** Training/index data is manipulated to create harmful behavior.
- **Model poisoning:** Weights are intentionally compromised.
- **Exfiltration:** Unauthorized extraction of data.
- **Secret:** Credential or sensitive token.
- **PII:** Personally identifiable information.
- **Redaction:** Removing or masking sensitive information.
- **Audit log:** Tamper-resistant record of important events.
- **Provenance:** Origin and transformation history of data or artifacts.
- **Hallucination:** Confident output not supported by model knowledge or evidence.
- **Groundedness:** Degree to which output follows supplied evidence.
- **Safe failure:** Refuses or stops rather than taking an uncertain dangerous action.

Required techniques:

- Treat retrieved text as untrusted data.

- Separate policy from prompts and model output.
- Apply strict schemas and resource limits.
- Default to read-only operations.
- Prevent path escape and symlink bypass.
- Scan training and retrieval data for secrets.
- Log decisions without logging sensitive content unnecessarily.

---

19. Common research and experimentation terms

- **Hypothesis:** Testable prediction.
- **Independent variable:** Component deliberately changed.
- **Dependent variable:** Measurement affected by the change.
- **Control/baseline:** Reference condition.
- **Confounder:** Uncontrolled factor that can explain a result.
- **Ablation study:** Removes or replaces components individually.
- **Scaling law:** Empirical relationship among model size, data, compute, and loss.
- **Parameter efficiency:** Quality achieved for a parameter budget.
- **Sample efficiency:** Quality achieved for a data budget.
- **Compute efficiency:** Quality achieved for a compute budget.
- **Pareto frontier:** Configurations where improving one objective worsens another.
- **Reproducibility:** Others can obtain the same result from recorded inputs.
- **Replication:** Independently repeating an experiment.
- **Seed sensitivity:** Results change meaningfully with initialization/data order.

For every architectural experiment, write:

```
Hypothesis:
Baseline:
Single change:
Fixed variables:
Metrics:
```

```
Expected result:
Observed result:
Decision:
```

---

20. What is essential now versus later

Level	Learn now	Learn later
Programming	Python, NumPy, PyTorch, tests, Git	Custom CUDA, distributed systems
Math	Shapes, matrix multiplication, probability, gradients	Advanced proofs, information geometry
Model	Embeddings, attention, FFN, norms, residuals	Mixture-of-experts, advanced SSM kernels
Training	Loss, backprop, AdamW, schedules, checkpoints	Large-scale distributed training
Data	Splits, cleaning, tokenization, deduplication	Web-scale data infrastructure
Product	Retrieval, schemas, constrained decoding, evaluation	Autonomous write tools

Topics explicitly not required before starting

- Reinforcement learning proofs.
- Multi-node distributed training.
- Tensor parallelism and pipeline parallelism.
- Custom CUDA kernels.
- Mixture-of-experts routing.
- Full-scale alignment pipelines.
- Billion-parameter model serving.

---

21. Recommended learning sequence

Stage A — Programming and tensors

- [ ] Python typing, classes, files, CLI, and tests.

- [ ] NumPy arrays, shapes, indexing, broadcasting, and matrix multiplication.
- [ ] PyTorch tensors, modules, autograd, and optimizers.

Build: A two-layer classifier and checkpoint loader.

Stage B — ML fundamentals

- [ ] Train/validation/test splits.
- [ ] Loss, gradient descent, AdamW, overfitting, and regularization.
- [ ] Metrics, baselines, seeds, and experiment tracking.

Build: A classifier that overfits a tiny dataset and generalizes on a clean split.

Stage C — Minimal language model

- [ ] Tokens, vocabulary, context, logits, softmax, and cross-entropy.
- [ ] Next-token prediction and autoregressive generation.
- [ ] Greedy, temperature, top-k, and top-p decoding.

Build: Byte-level bigram language model.

Stage D — Transformer

- [ ] Embeddings, causal masks, Q/K/V, multi-head attention.
- [ ] RMSNorm, residuals, feed-forward network, and RoPE.
- [ ] KV cache and optimized attention.

Build: Small decoder-only Transformer with leakage tests.

Stage E — Data and training engineering

- [ ] BPE, manifests, cleaning, deduplication, packing, and memory maps.
- [ ] Mixed precision, accumulation, checkpointing, and profiling.

Build: Reproducible tokenizer-to-checkpoint pipeline.

Stage F — PocketMind-specific architecture

- [ ] Local attention, recurrence, gates, state isolation, and ablation.
- [ ] Retrieval cross-attention and auxiliary objectives.

Build: Hybrid model and equal-parameter baseline comparison.

Stage G — Practical system

- [ ] Chunking, BM25/TF-IDF, dense retrieval, reranking, and RAG.
- [ ] JSON schema, constrained decoding, policy, executor, and prompt injection.
- [ ] Grounding, abstention, and end-to-end evaluation.

Build: Read-only local project assistant.

---

22. Beginner experiments before the full build

Experiment 1 — Learn a line

Train a tiny model to memorize one short text. Learn tokenization, shifted targets, loss, backward, and generation.

Experiment 2 — Visualize attention

Use a five-token sequence, print the causal mask, raw scores, probabilities, and weighted values.

Experiment 3 — Break causality deliberately

Remove the causal mask, observe suspiciously low training loss, then restore the mask and add a leakage regression test.

Experiment 4 — Compare tokenizers

Encode the same code/document set with bytes and BPE. Measure vocabulary size, average sequence length, and round-trip correctness.

Experiment 5 — Retrieve before generating

Implement document search without a neural model. This establishes a retrieval baseline and demonstrates that system usefulness does not depend entirely on model size.

Experiment 6 — Constrain JSON

Generate JSON one character or byte at a time using a parser state machine. Prove invalid structural tokens cannot be selected.

Experiment 7 — Resume training

Train for a few steps, save, resume, and compare against an uninterrupted run.

---

23. Questions you should be able to answer

Before implementing the tokenizer

- Why use bytes as the base alphabet?
- How does vocabulary size affect sequence length and parameter count?
- What does round-trip correctness mean?

Before implementing attention

- What are the shapes of Q, K, V, scores, and output?
- Why divide by square root of head dimension?
- What does the causal mask prevent?
- How can a test detect future-token leakage?

Before training

- What is the target for every input position?
- Why separate training, validation, and test data?
- What do loss, perplexity, gradient norm, and learning rate tell you?
- Why must a tiny model overfit one batch?

Before adding recurrent memory

- When is hidden state reset?
- How can state leak across examples?
- What baseline proves the recurrent layer helps?

Before adding retrieval

- Is a bad answer caused by retrieval or generation?
- What is Recall@k?
- How are chunks formed and attributed?
- How should the model behave when evidence is missing?

Before enabling actions

- What guarantees syntactic validity?
- What guarantees policy authorization?
- Why can’t validation alone replace access control?
- How are path traversal, symlinks, timeouts, and output size handled?

---

24. Compact glossary of project configuration names

- `vocab_size`: Number of token types.
- `context_length`: Maximum tokens processed in one sequence.
- `d_model`: Main hidden-vector width.
- `n_layers`: Number of sequence-processing blocks.
- `n_heads`: Attention heads per attention layer.
- `head_dim`: Width of each attention head.
- `ffn_dim`: Hidden width inside the feed-forward network.
- `dropout`: Probability of dropping eligible activations during training.
- `attention_window`: Number of nearby positions visible to local attention.
- `recurrent_state_dim`: Width of recurrent hidden state.
- `micro_batch_size`: Examples processed per forward/backward pass.
- `gradient_accumulation`: Micro-batches combined before an optimizer update.
- `max_steps`: Maximum optimizer updates.
- `learning_rate`: Base update scale.
- `weight_decay`: Regularization applied to eligible weights.
- `warmup_steps`: Steps spent increasing learning rate.
- `max_new_tokens`: Maximum generated output tokens.
- `temperature`: Sampling randomness control.
- `top_k`: Highest-scoring candidate count retained.
- `top_p`: Cumulative probability threshold retained.

25. Final prerequisite checklist

Must understand

- [ ] Tensor shapes and matrix multiplication.
- [ ] Softmax and cross-entropy.
- [ ] Gradients, backpropagation, and AdamW.
- [ ] Training, validation, testing, overfitting, and leakage.
- [ ] Tokens, embeddings, logits, and autoregressive generation.
- [ ] Causal self-attention and masks.
- [ ] Normalization, residual connections, and feed-forward networks.
- [ ] Checkpoints, seeds, mixed precision, and gradient accumulation.
- [ ] Corpus cleaning, deduplication, tokenization, and memory mapping.
- [ ] Retrieval, chunking, ranking, grounding, and abstention.
- [ ] Schemas, constrained decoding, validation, and least privilege.
- [ ] Metrics, baselines, ablations, and reproducibility.

Must be able to implement before the full model

- [ ] Stable softmax.
- [ ] Cross-entropy example.
- [ ] Byte tokenizer.
- [ ] Memory-mapped shifted-token dataset.
- [ ] Two-layer PyTorch network.
- [ ] Bigram language model.
- [ ] Single causal-attention head.
- [ ] Save/reload checkpoint test.
- [ ] Tiny-overfit test.
- [ ] Basic lexical retriever.
- [ ] Strict typed JSON action schema.

Ready-to-build rule

You are ready to start the full PocketMind roadmap when you can:

1. Explain the complete path from text to loss and from prompt to generated token.
2. Track the shape of every tensor through one attention block.
3. Intentionally overfit a tiny batch and explain why that test matters.
4. Save and resume training without losing required state.
5. Explain why retrieval and deterministic policy make a small model more useful and safer.

You do not need to memorize this glossary. Use it as a map while building. Learn each term when its corresponding implementation phase begins, then prove understanding with a small experiment and a test.